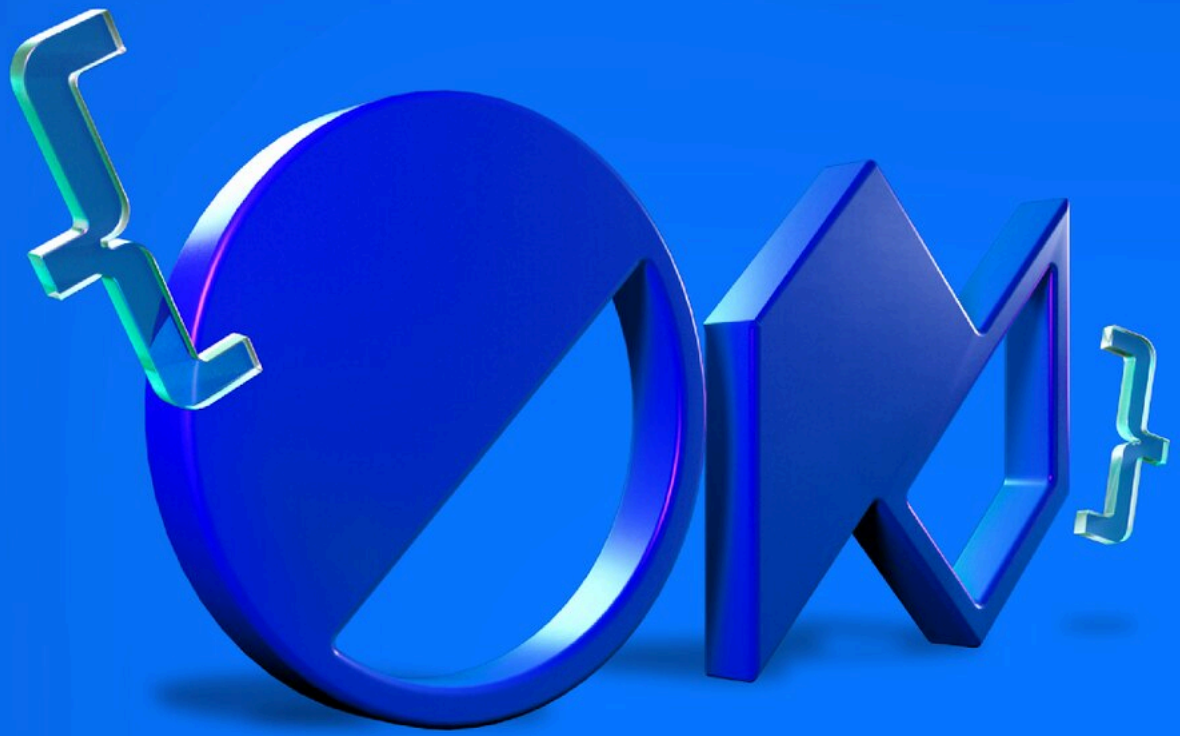
Search is test-time compute

ElasticON AI San Francisco, November 4, 2026

Han Xiao, VP of AI, Elastic



elastic

Search has mostly improved by **training harder**.

Encoders, data, indexes and rerankers all improve before the first query.



Recent LLMs found a second axis: compute at inference



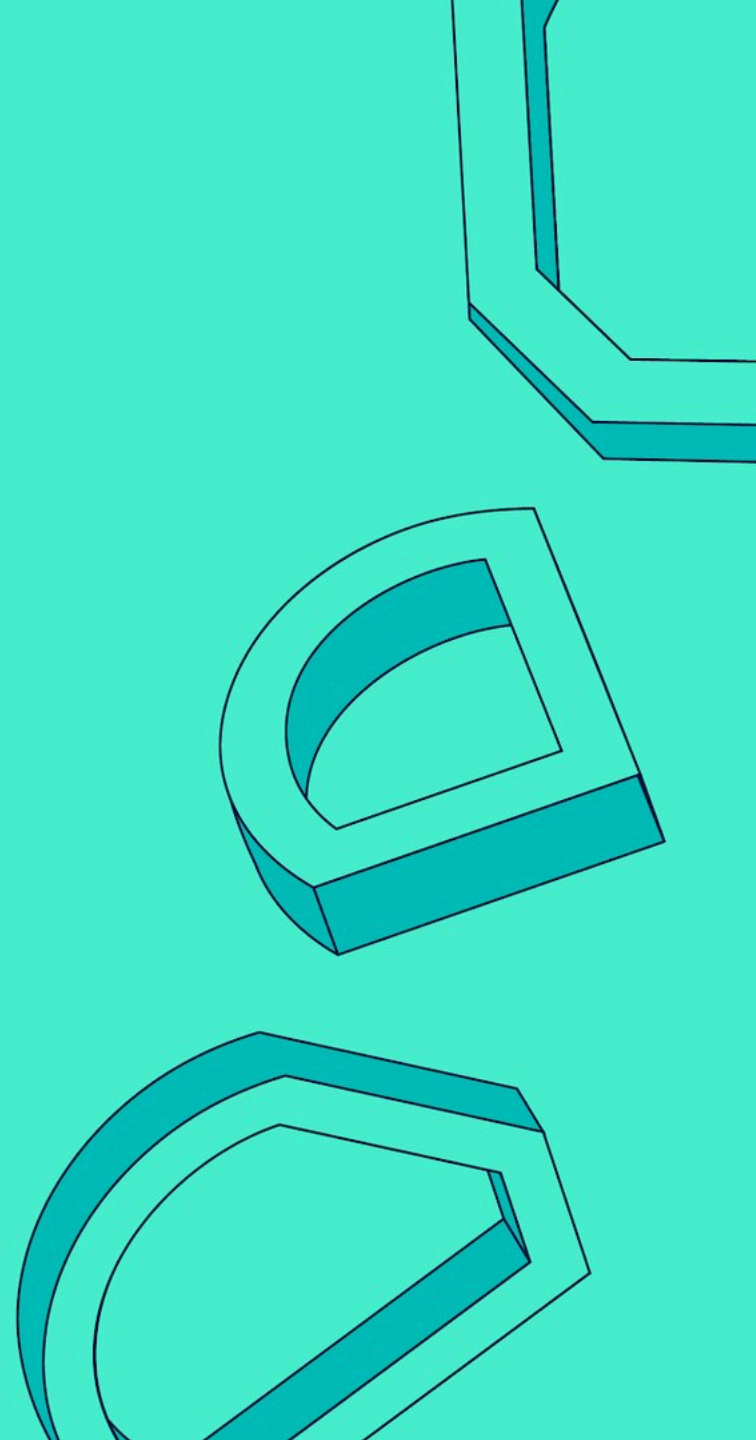
> Having a bot think for just **20 seconds** in a hand of poker got the same performance boost as scaling up the model by **100,000x** and training it for 100,000 times longer.

Noam Brown, OpenAI, 2024

elastic

Fun fact: search has always lived on **that axis**.

Embedding, retrieval, reranking, expansion and fusion all run after the user presses Enter.



Inference time is a currency, not a KPI

Until now: a KPI

Inference time is a KPI

Latency is a limit written into the SLA. **Better accuracy requires a bigger model.**

becomes

In this talk: a currency

Inference time is a currency

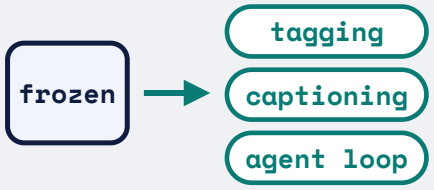
Keep the weights and raise the budget from 10 ms to 100 ms. What does it buy?



buys

Relevance

Better ranking on the same task.

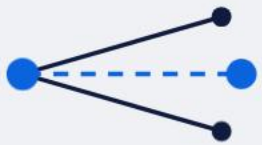


buys

A new capability

Tasks the model was never trained for.

Four training-free demos



01

Score functions

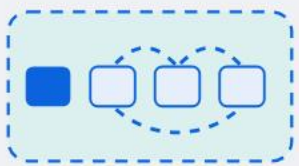
Higher relevance from the same frozen encoder.



02

Open-vocabulary tagger

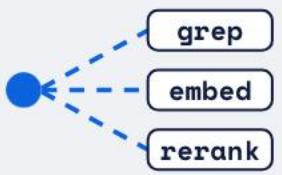
Annotate photos with any words.



03

Semantic highlighting

A reranker highlights the passages that match.



04

Reusable search pipelines

An agent builds small pipelines and reuses them in long-horizon tasks.

elastic

Three ways to spend compute on a frozen model

A Deeper pass



Read every token or patch vector of the pass you already ran.

New information: from inside the pass

B More passes



Run the encoder again on new views, such as sentences or image crops.

New information: from the input

C Calibration



Transform existing vectors with whitening, label propagation or optimal transport.

New information: from a global prior

elastic



01

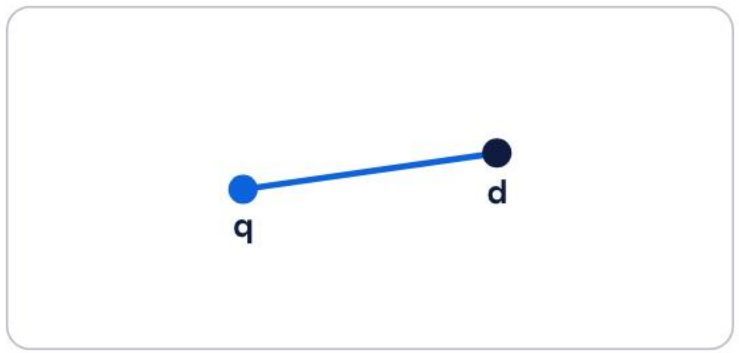
Score functions on a frozen encoder



elastic

Score function defines the relevance score

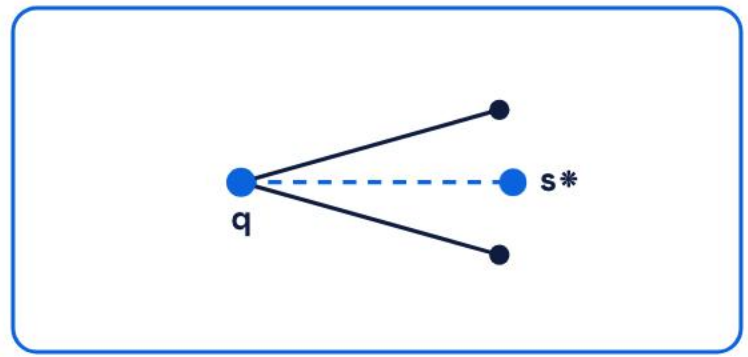
single-vector cosine



$\cos(q,\ d)$

one vector per document

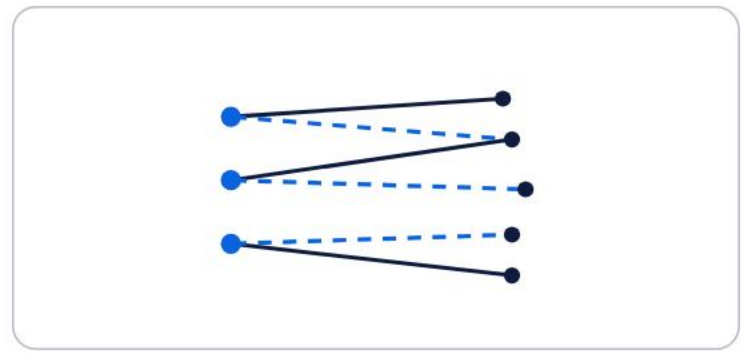
sentence MaxSim



$\max_s\ \cos(q,\ s)$

same encoder, one vector per sentence

ColBERT multi-vector

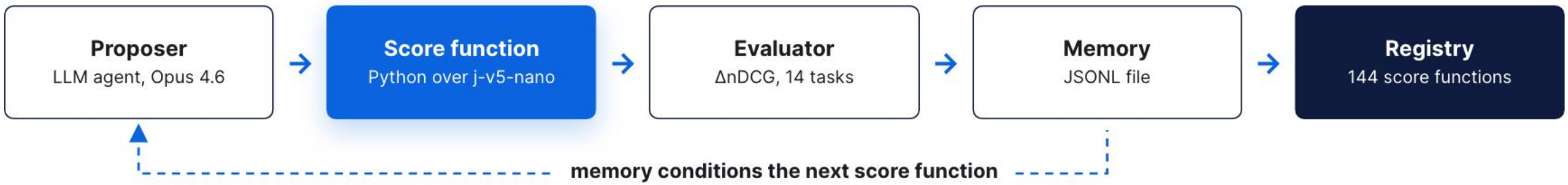


$\sum_i \max_j\ \cos(q_i,\ d_j)$

max over document tokens for each query token

elastic

An LLM agent writes score functions over the frozen encoder



writes	encoder	scores	logs	keeps
one score function per round	239M frozen	14 MMTEB tasks ΔnDCG@10	Δ, cost, parent and lesson	144 score functions

The evaluator rejects hyperparameters, task-specific routing, external models and learned parameters.

elastic

We search on one encoder and test on three others

Discovery · the loop searches here

`j-v5-nano`
239M · Jina
discovery encoder

small and fast, so each round is quick

searched over
14 MMTEB Tier-1 tasks · legal, financial, long-doc, general



the same score function

Held-out · the loop never sees these

`j-v5-small`	`gemma-300m`	`qwen3-0.6b`
568M · Jina	303M · Gemma 3	600M · Qwen3
same family	**unseen family**	**unseen family**

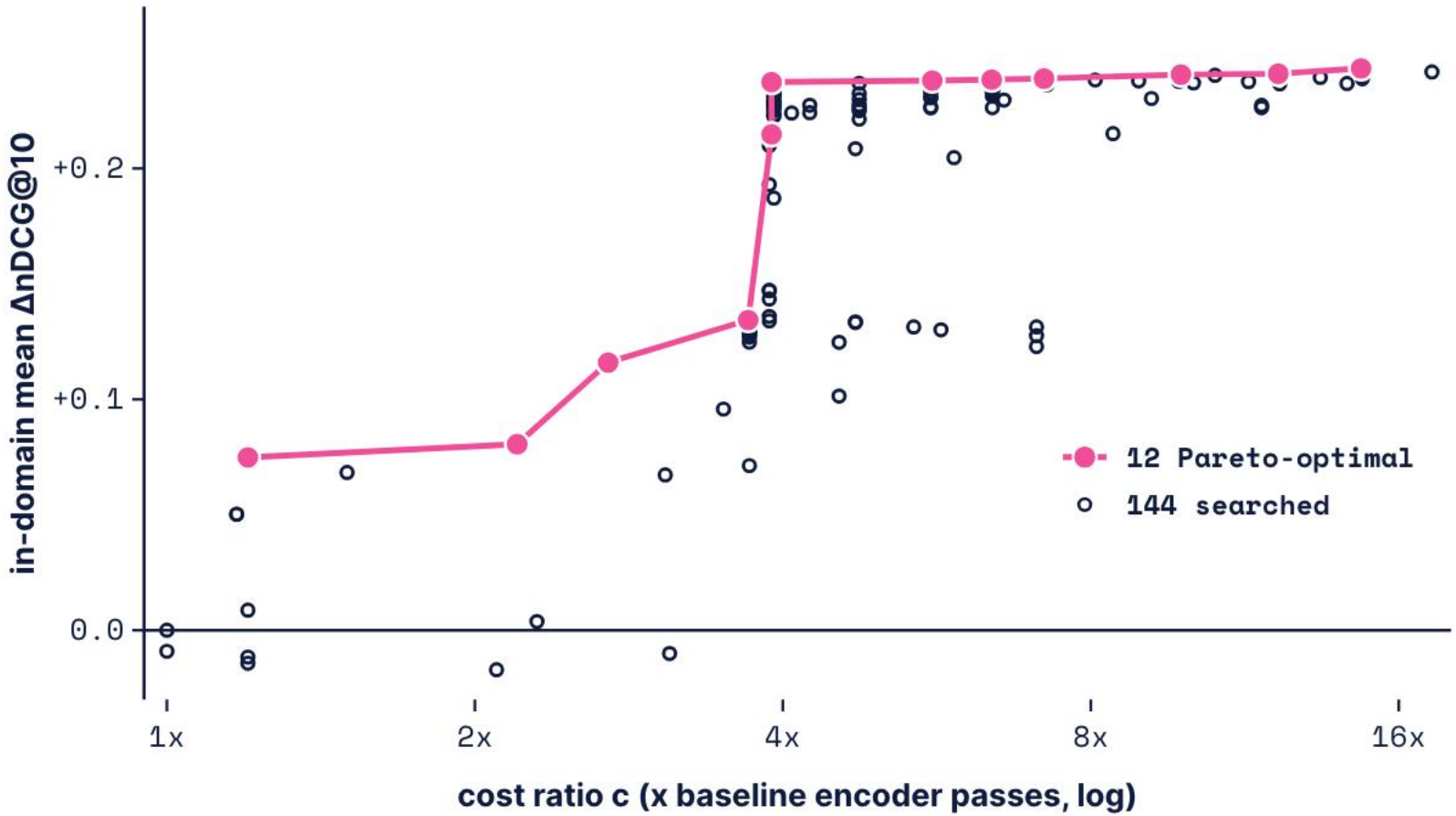
unseen families test transfer

tested on
19 MMTEB Tier-2/3 tasks · summarization, QA, fact-check, ...

Metric: ΔnDCG@10 against the cosine baseline. **Cost c:** encoder passes per query, relative to the baseline.

elastic

Rewarding compute gives a Pareto curve



144

score functions searched

12

Pareto-optimal, from c = 1.2 to 14.7

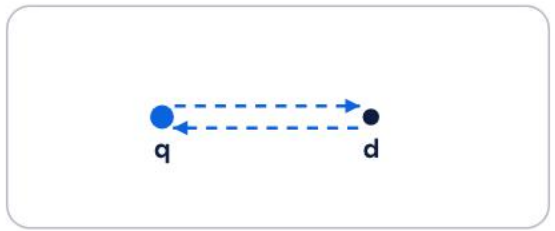
+0.24

best in-domain mean ΔnDCG@10

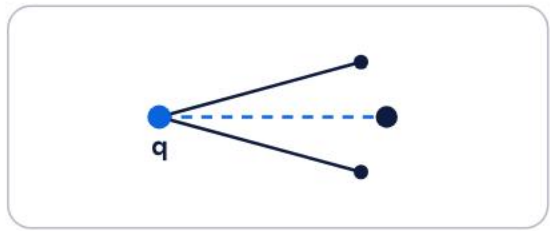
Measured on the 14 discovery tasks.

elastic

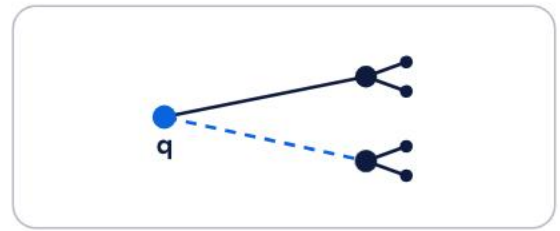
Twelve score functions on the compute frontier



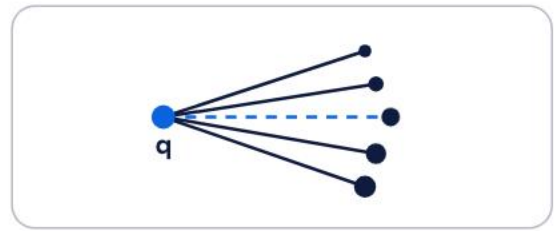
BidirZScore c=1.2



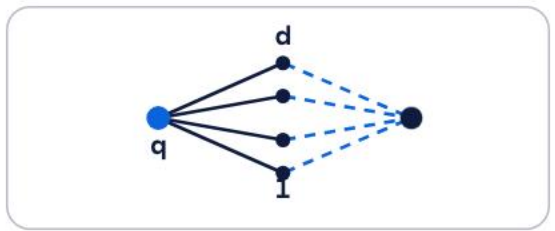
SentMaxSim c=2.2



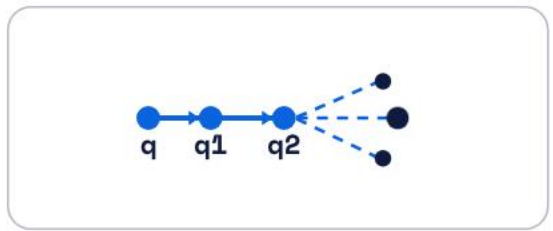
AdaptGranularity c=2.7



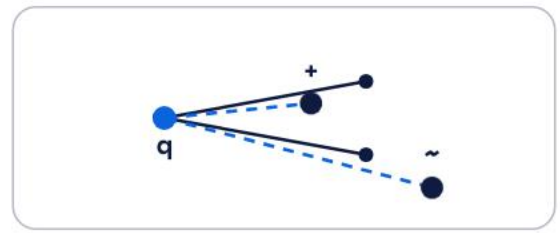
CoverageTriple c=3.7



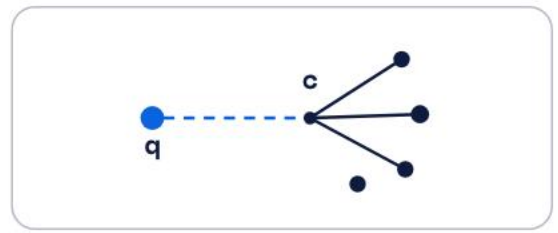
LexicalHybridRRF c=3.9



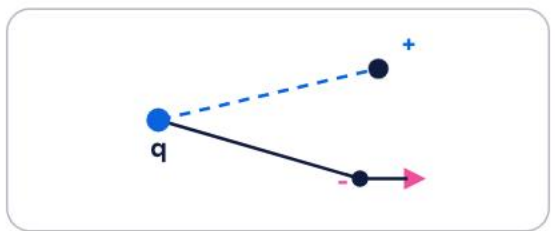
CrossRoundRRF c=3.9



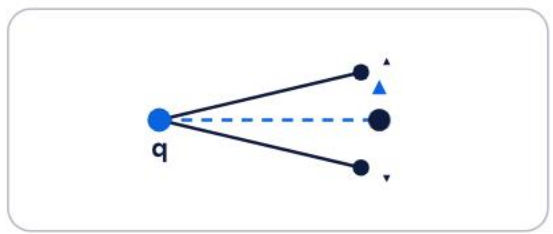
DiverseDualCtx c=5.6



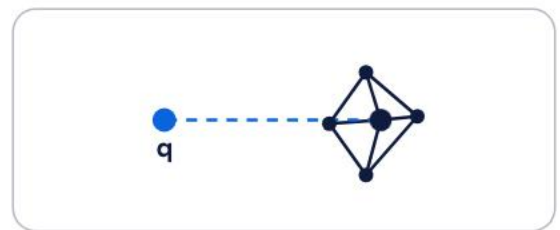
ConsensusRocchio c=6.4



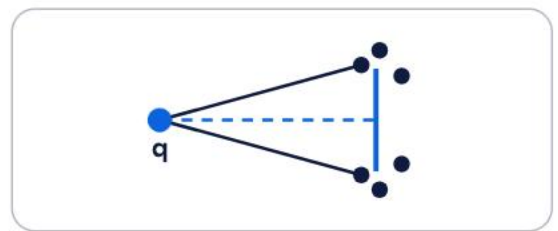
NegContrastive c=7.2



MomentumProg c=9.8



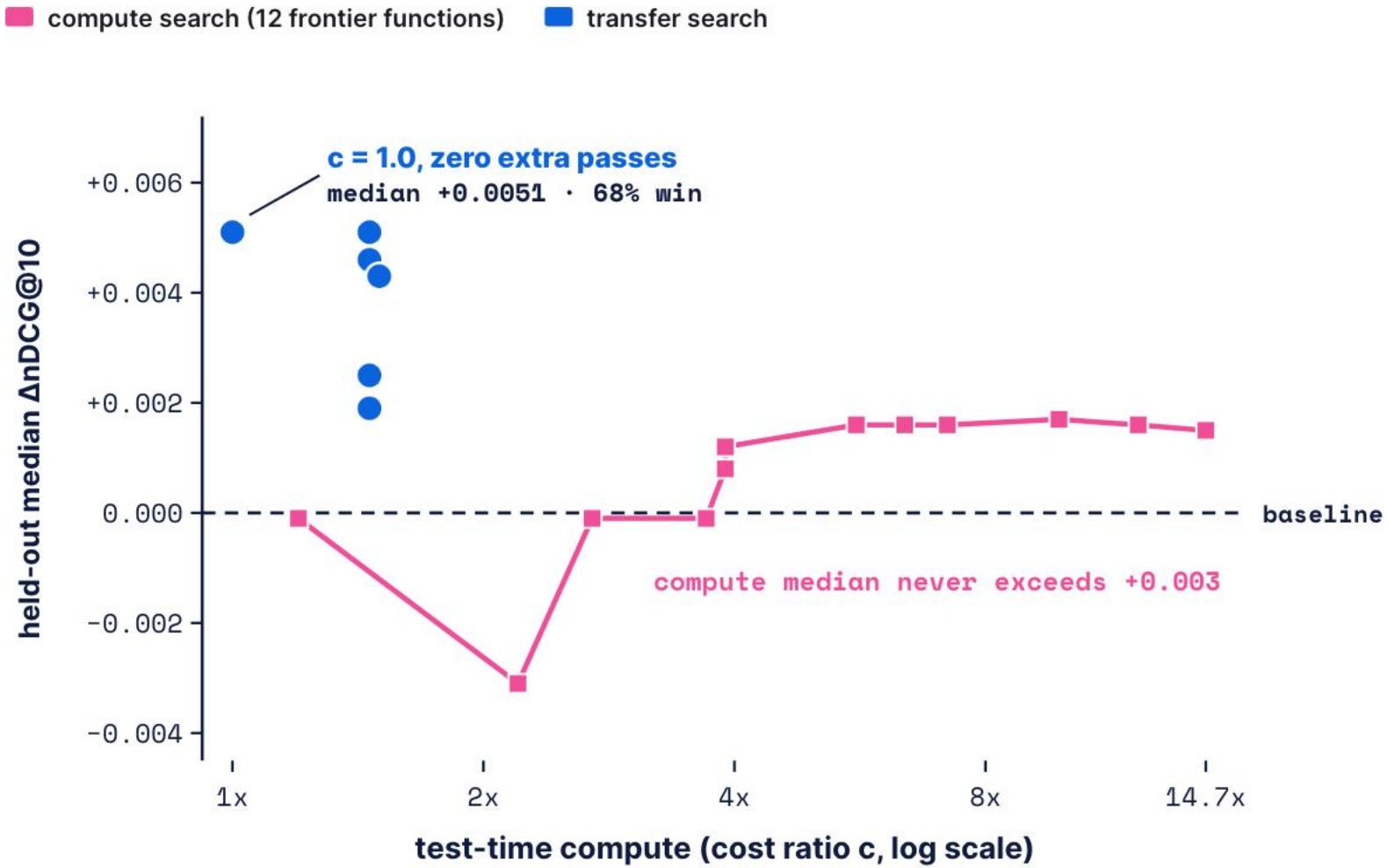
GraphCentrality c=12.2



FisherStability c=14.7

elastic

On held-out tasks, a score function at c = 1.0 beats all 12 frontier functions



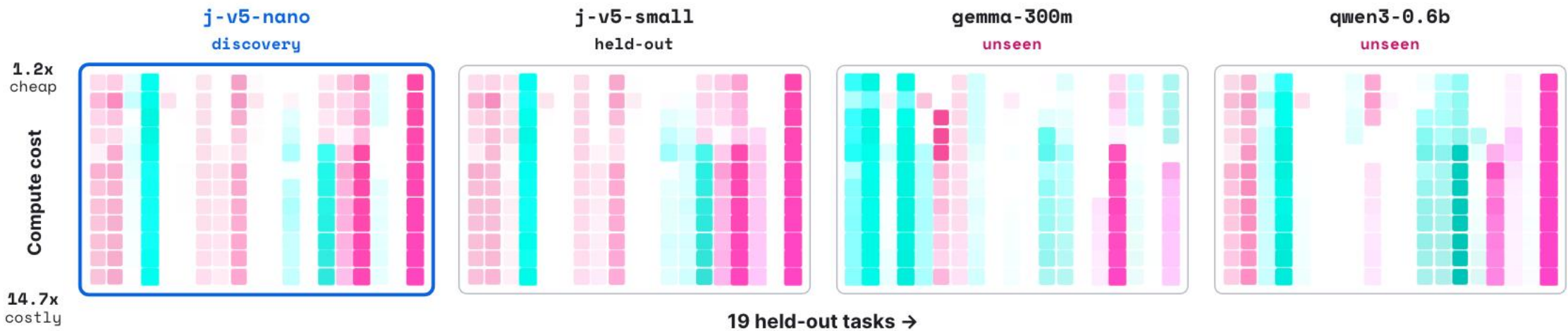
+0.003

highest held-out median of the compute frontier, up to 14.7x

+0.0051

held-out median at c = 1.0, with a 68% win rate

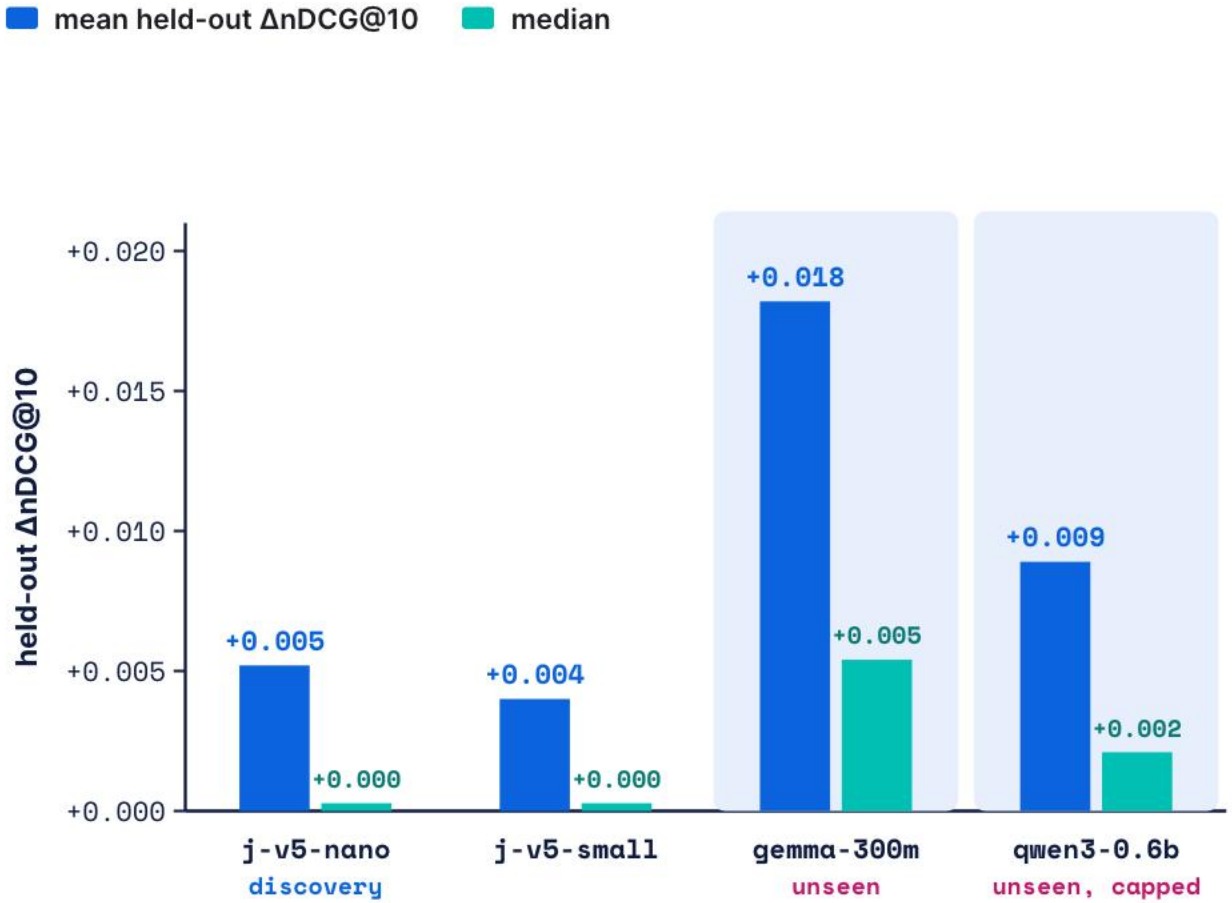
Compute improves about half the cells and badly hurts others



52 of 76 task-encoder pairs improve under at least one score function.

elastic

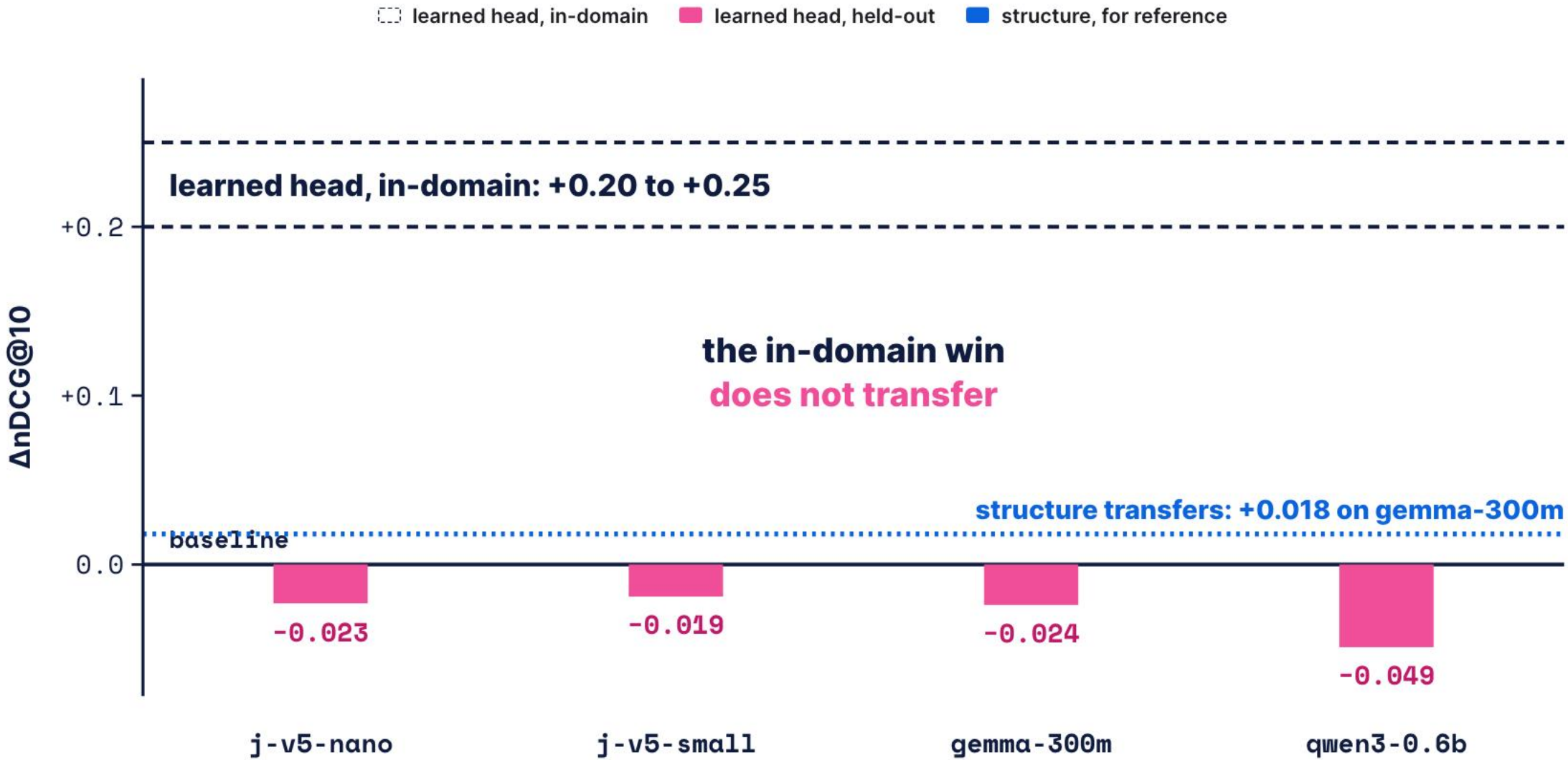
A transfer search finds cheap score functions that generalize



- Mean gains are positive on all four encoders and largest on unseen families.
- Transpose-Consensus never loses a task: **83% win rate**, worst cell **0.000**.
- Applied unmodified to French and Greek: median **+0.016**, **86%** win rate.

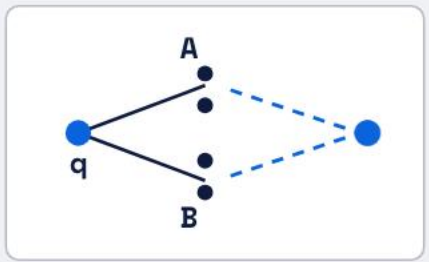
elastic

A learned head at the same data budget memorizes



elastic

The search re-derives classical IR in embedding space



Reciprocal Rank Fusion

appeared without a seed

fuse two rankings into one



Fisher Linear Discriminant

appeared without a seed

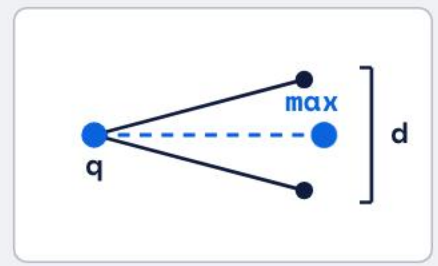
the axis that best separates relevant and non-relevant



Rocchio Pseudo-Relevance Feedback

operationalized from a seeded idea

pull the query toward the relevant centroid



Sentence-level MaxSim

operationalized from a seeded idea

score a document by its best sentence

elastic

02

A retrieval model tags images

Open-vocabulary tags from a frozen embedding model



9 photos · top-5 tags each

elastic

Images and text share one embedding space



per image · test time

cat
coche
кот

× 128,260 tokens

vocabulary · encoded once

jina-embeddings-v5-omni-nano

Qwen3-VL vision tower
16 px patches
2×2 merge

EuroBERT-12L text tower
bidirectional · 768-d

vision tokens injected at image-token positions

frozen

P: one 768-d row per patch, attending to the *whole* image

g: pooled image vector

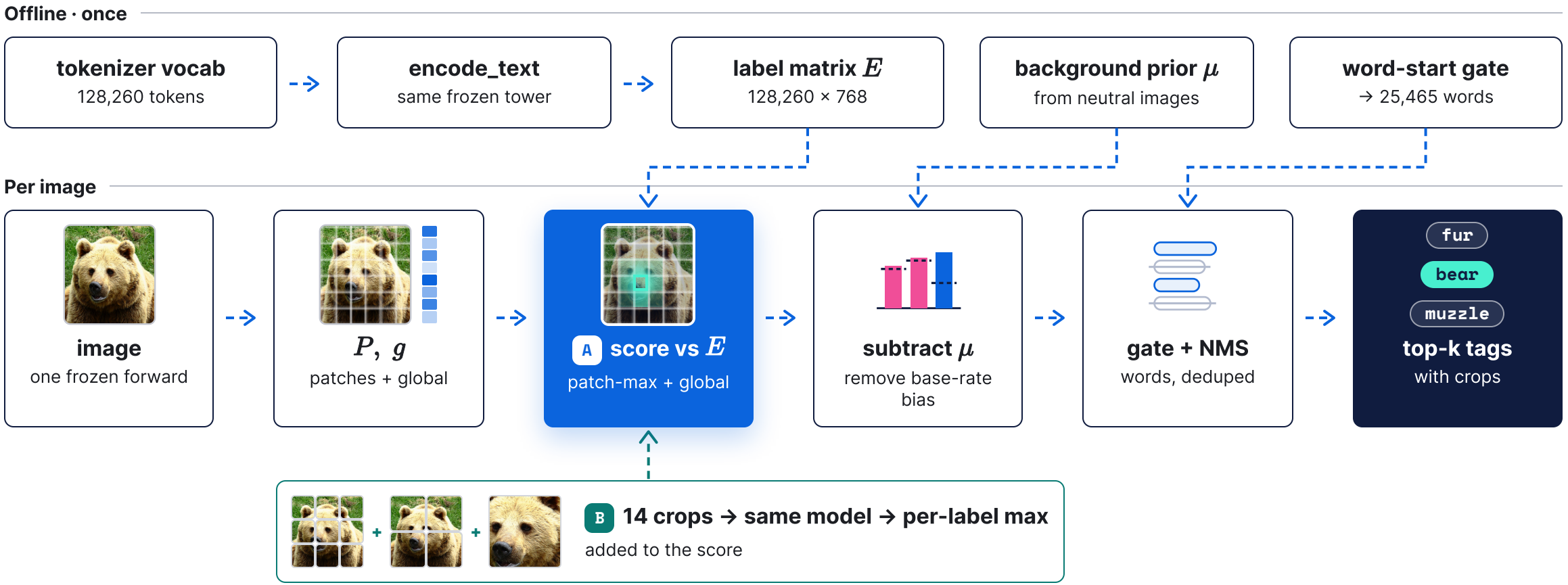
E: label matrix, 128,260 × 768

Any image patch can be scored against any word in the vocabulary.

elastic

The whole tagger is test-time compute

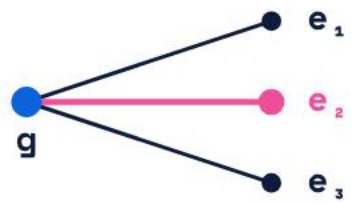
The label set is the tokenizer vocabulary: 128,260 tokens, gated to 25,465 whole words.



Every box is the frozen model or arithmetic. A reads more of one pass, B adds passes on crops.

Scoring every patch lifts mAP from 0.264 to 0.635

global pooled

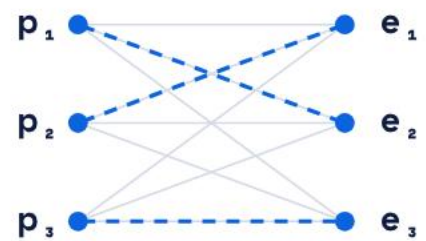


$$s_\ell = \langle g,\, e_\ell \rangle$$

one vector, dominated by the most salient object

mAP 0.264

per-label max over patches



$$s_\ell = 0.7 \max_{p \in P} \langle p, e_\ell \rangle + 0.3 \, \langle g, e_\ell \rangle$$

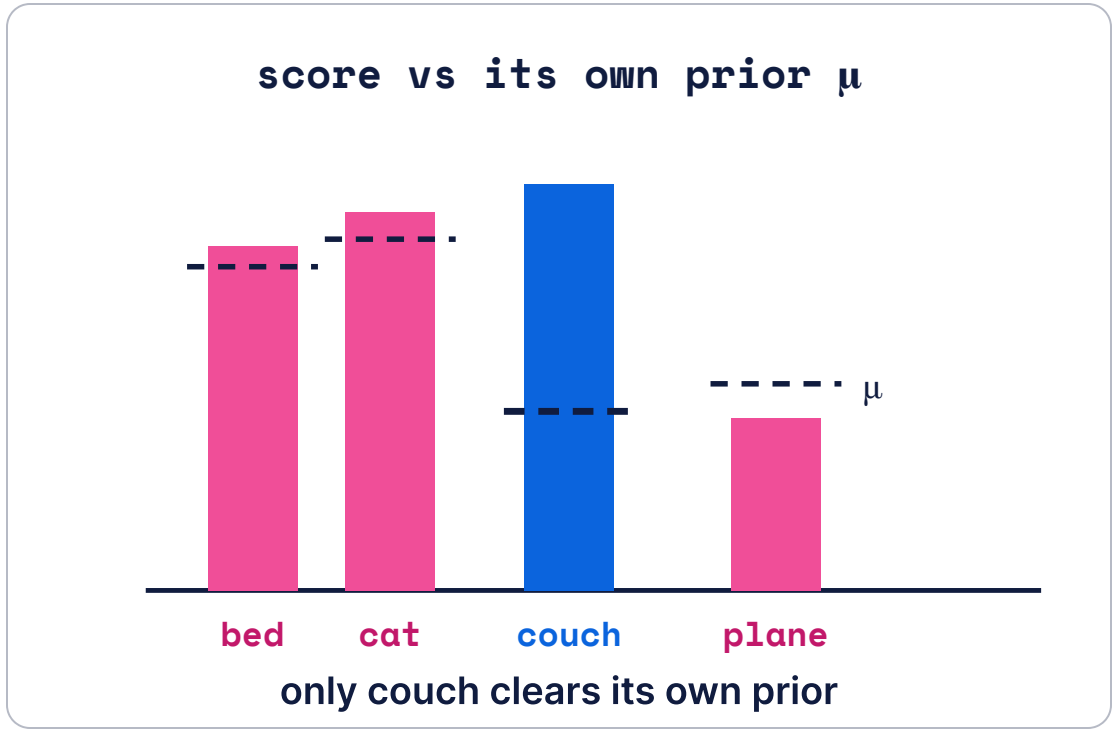
best patch per label, plus the global vector

mAP 0.635

elastic

Each label is compared with its own prior

Generic words such as bed and cat score high on every image. A prior from neutral images removes that bias.



- One offline pass, with no calibration set or labels.
- **Word-start gate:** keep tokens that the tokenizer marks as word starts.
- **Embedding-NMS:** drop any word within cosine 0.6 of one already kept, so cat, kitty and кот merge.

Crops give the model new pixels



- **+0.075 mAP** for 14x the passes: 0.635 to 0.710.
- P@1 goes from **0.753** to **0.813**.
- One pass reads **wolf, roar, muzzle**. With crops it reads **fur, bear, muzzle**.

elastic

The whole tagger is one recursive scoring function

Recursion: the same scoring function runs again on each crop as a new image

$$S(\ell) = \underbrace{0.3\left(\langle g,\, e_\ell\rangle - \mu_\ell^g\right)}_{\text{C Calibration}} + \underbrace{0.7\left(\max_{p\in P}\langle p,\, e_\ell\rangle - \mu_\ell\right)}_{\text{A Patch evidence}} + \underbrace{1.3\left(\max_{c=1..14} s_\ell(\text{crop}_c) - \mu_\ell\right)}_{\text{B Multi-crop re-encode}}$$

C Calibration
The pooled vector minus its own background prior, at no extra cost.

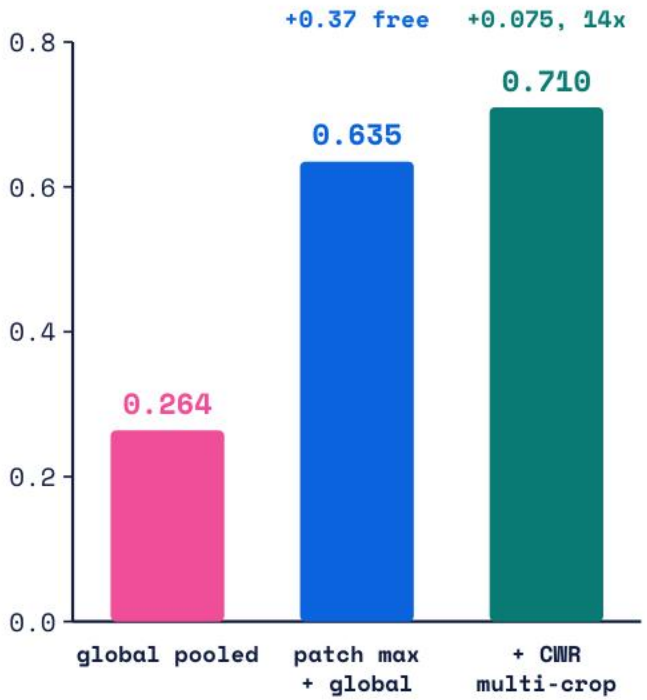
A Deeper pass
Arithmetic on patch rows the forward pass already produced, at no extra cost.

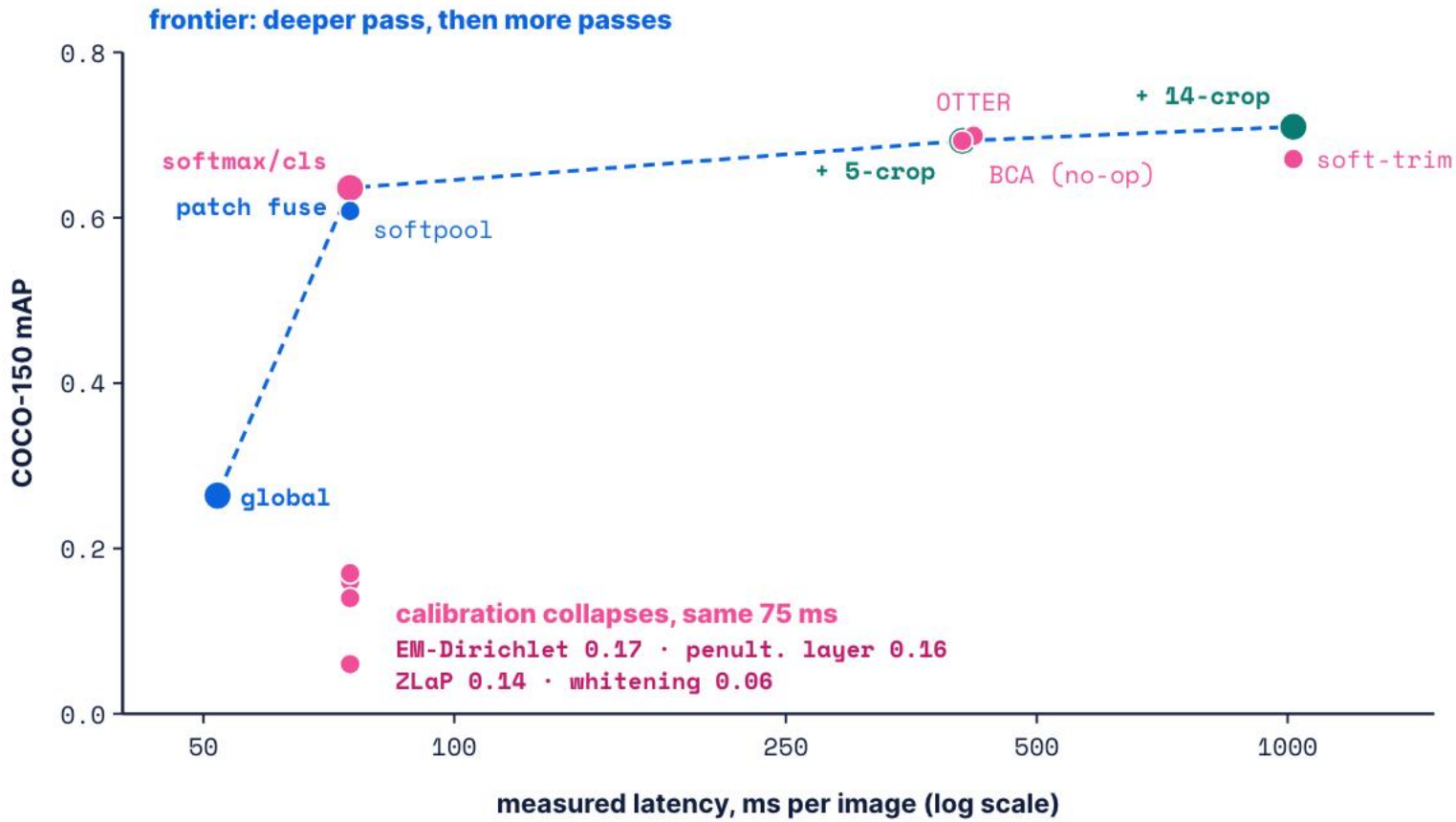
B More passes
14 new forward passes over new pixels.

$\ell \in$ word-start gated vocabulary $\rightarrow$ embedding-space NMS $(\tau = 0.6)$ $\rightarrow$ top-k

The tagger has no classifier head, logits or learned thresholds. It uses three fixed weights, two background priors and a few max operations.

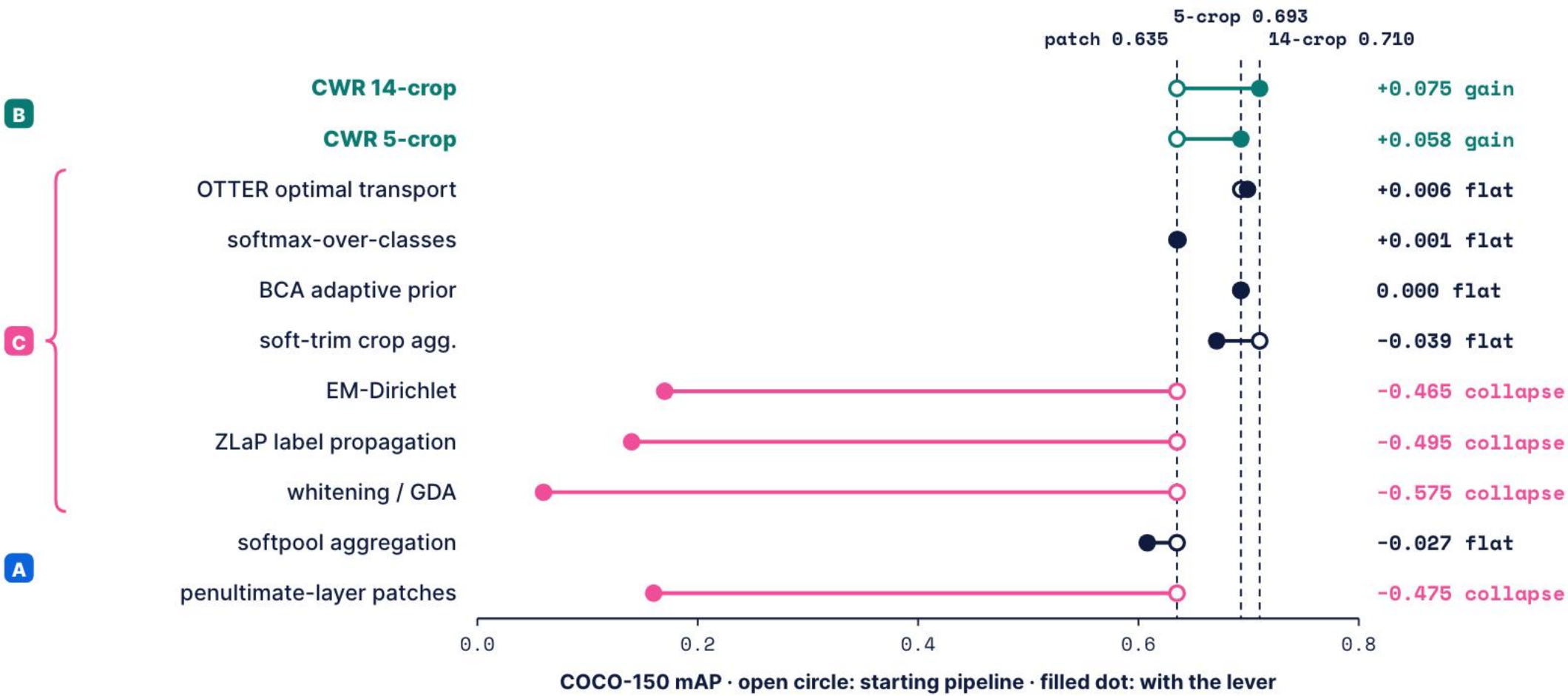
Accuracy climbs with each kind of new information





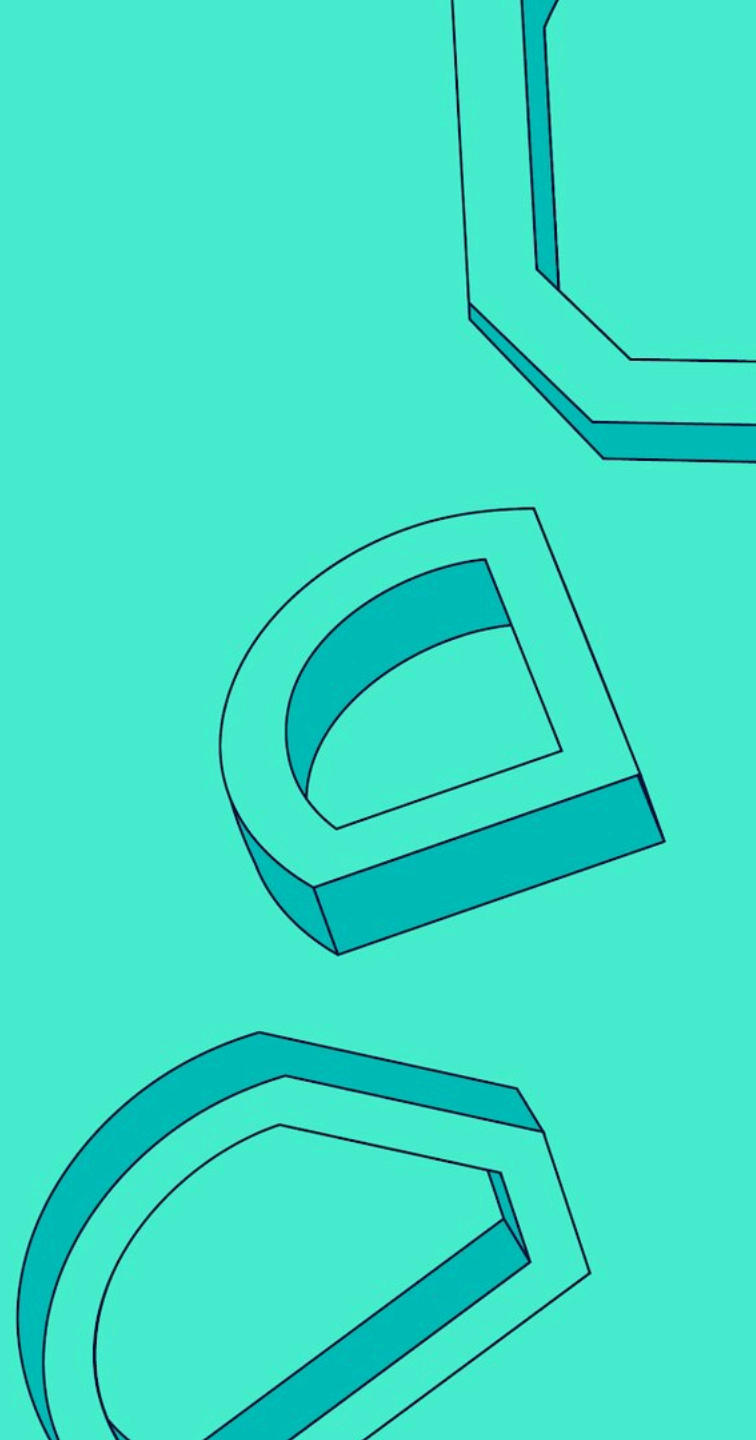
COCO-150 mAP: global → patch → + 14 crops

Calibration does not move accuracy



Only **new information** moves accuracy.

Patches and crops add information. Calibration rearranges what is already there.



omni-macos

Tagging runs on device in Omni

~0.6 ms

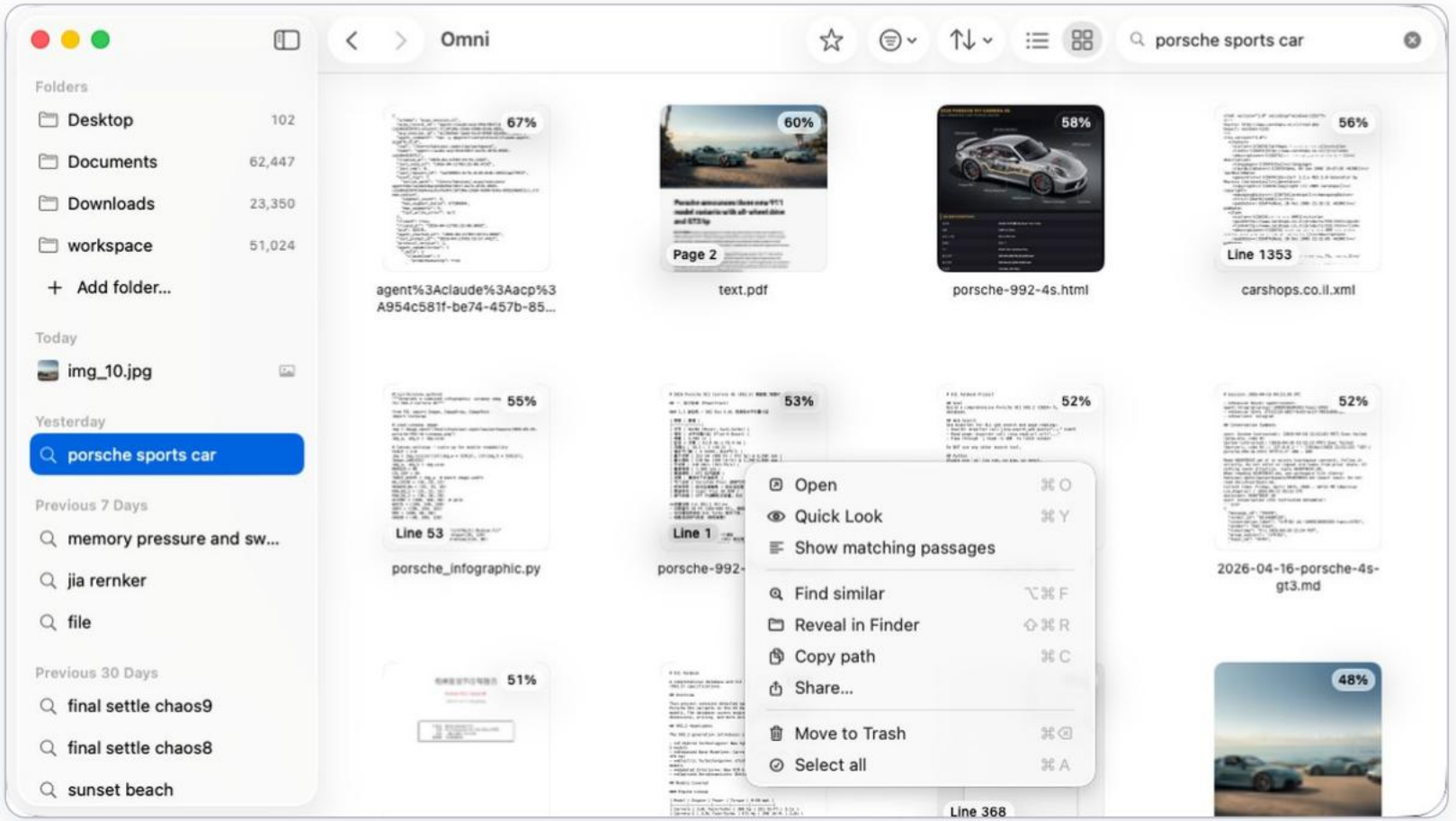
per image

~4%

of the embedding step

64

images calibrate the prior

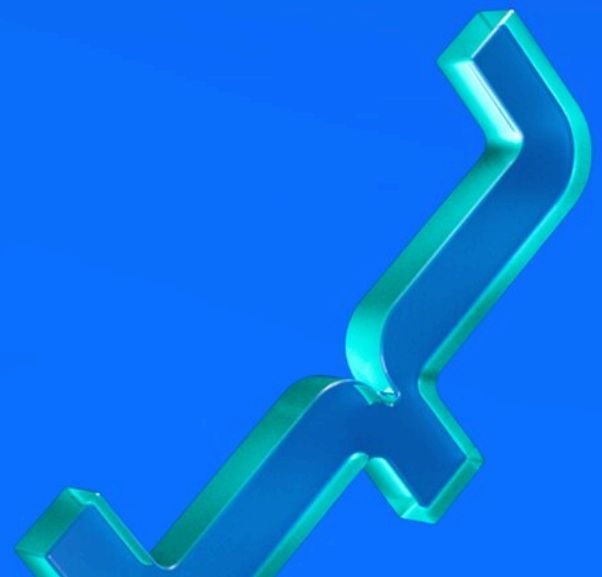


elastic

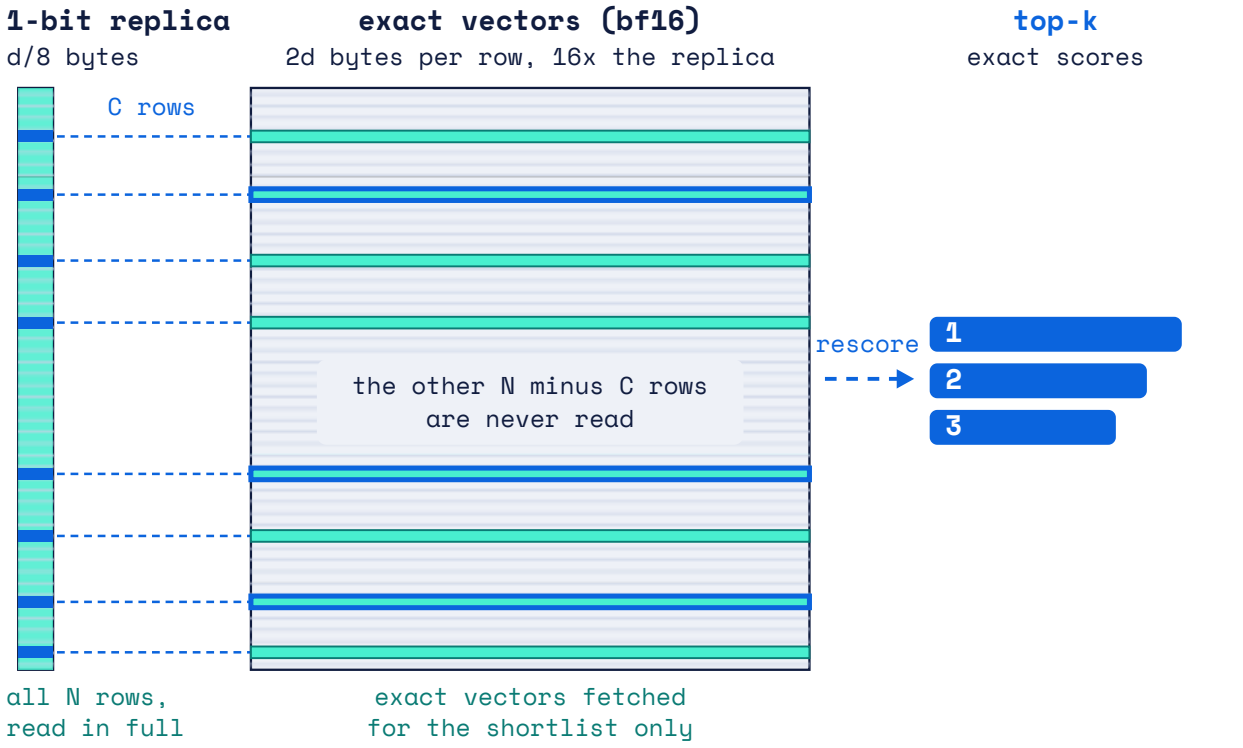


03

One call replaces an index



Omni builds no index and spends compute on rescoring



01 SCAN	02 SHORTLIST	03 RESCORE	04 TOP-K
every row, 1 bit	top C, coarse	exact, C rows	exact order

Teal marks what one query reads.

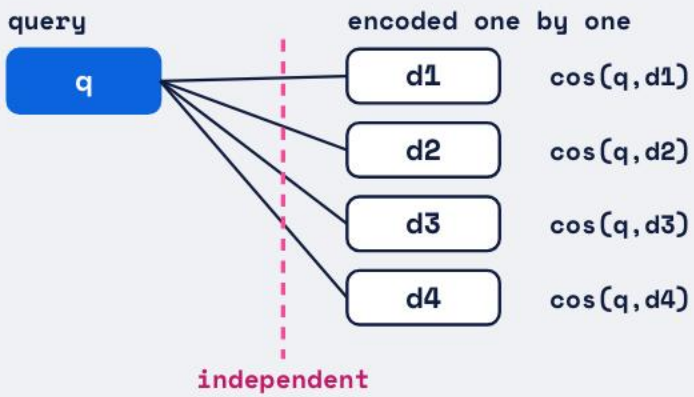
Every query scans the whole corpus. A graph index would add 256 bytes of links per row, **0.95 GB at four million chunks**, close to three times the scanned matrix, and edits keep invalidating it.

The scan reads a 1-bit replica. Exact vectors are read only for the shortlist and set the final order.

Shortlist size: `min(4096, max(1024, topK×32))`

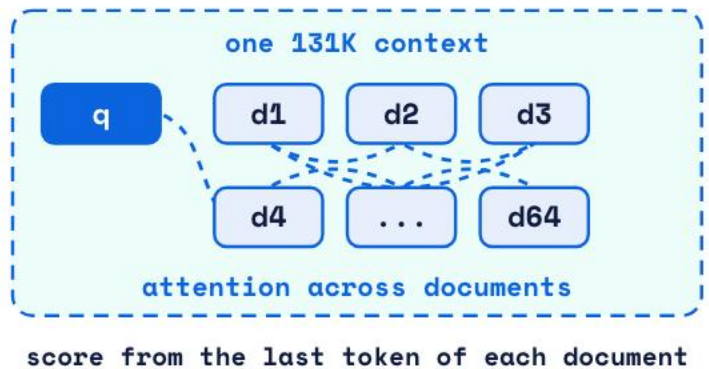
Let the candidates see each other in one forward pass

Bi-encoder each document scored alone



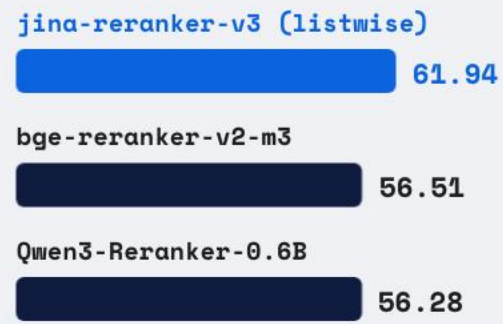
Each document vector is computed without the others.

Listwise all candidates in one context



Each document is scored after attending to every other candidate.

BEIR nDCG@10 0.6B class



At the same size, the listwise model leads.

elastic

GitHub

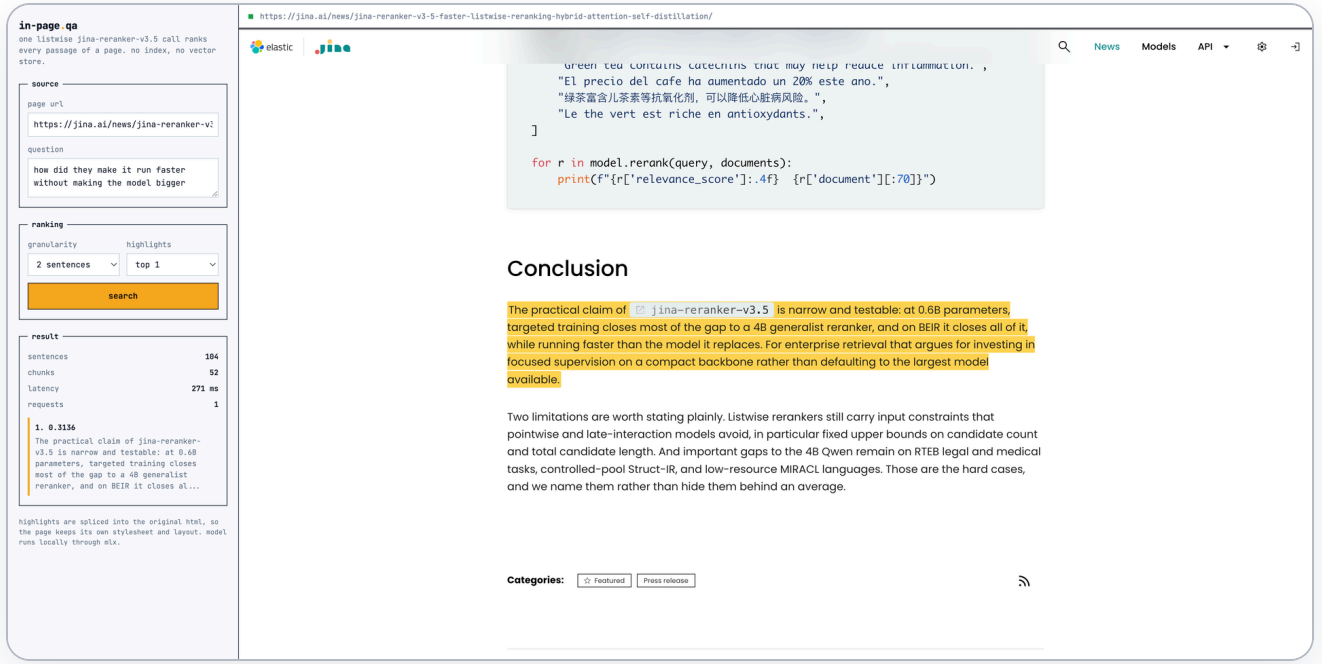
A single page needs no index

An index pays off over many queries. A page lives for one visit.

- jina-reranker-v3.5 scores a 53-chunk page in **143 ms** per call.
- Answers are highlighted in place on the original page.

ms per call · 53-chunk page

jina-reranker-v3.5	143
jina-reranker-v2-base-multilingual	4,194



04

An agent builds the pipeline

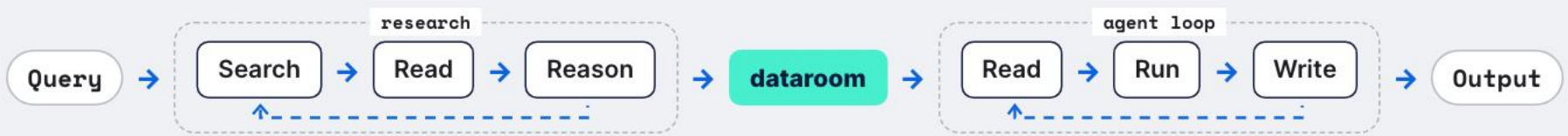
Test-time compute is the pattern for long-horizon agents

2025 · Deep research



one loop · web IO throughout

2026 · Long-horizon tasks



research with web IO → corpus → offline agent loop, no web IO

elastic

dataroom: a token budget builds a corpus

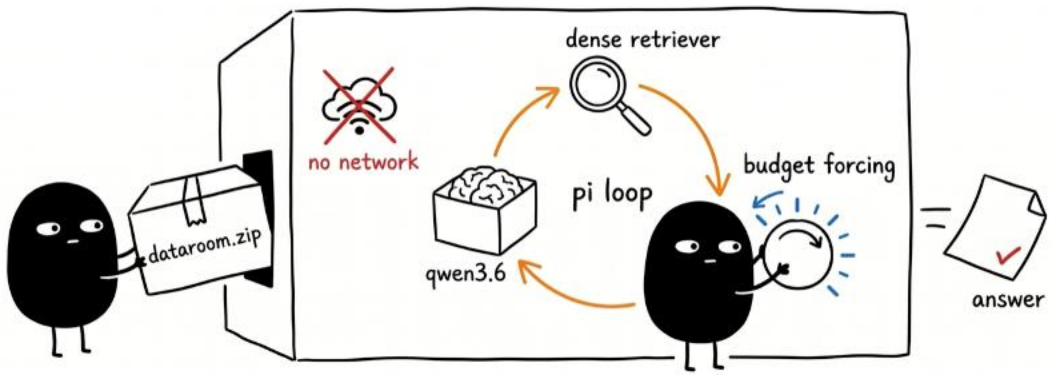
GitHub



- It stops at a measured coverage floor.
- The job page tracks coverage, tokens and tool calls.
- The .zip goes to searchbox or to a frontier model.

searchbox: an agent builds its pipeline for each query

GitHub



- **Tools:** grep, embed, rerank, similarity, cluster, select_diverse.

→ Which tool does the agent call first?

→ When does grep match a dense retriever?

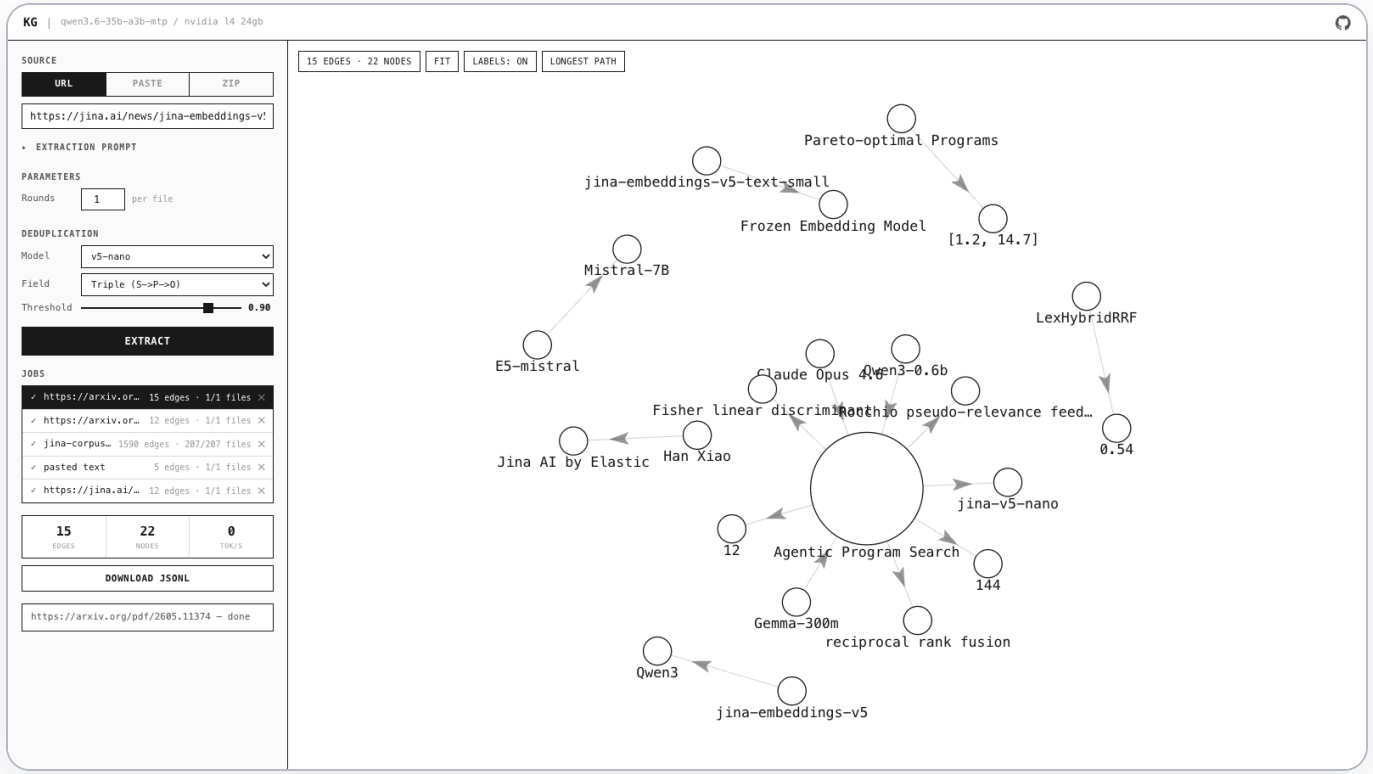
→ Does a larger token budget help on the hard questions?

elastic

GitHub

knowledge-graph: hard questions as a private verifier

If one grep finds the answer, every method scores the same. Agentic search needs questions that force several hops.



- Turn the corpus into subject-predicate-object edges.
- Walk paths of length 2 and 3, hide the bridge entities and ask for the endpoint.
- Keep a question only if a closed-book probe fails and a retrieval agent succeeds.

Lessons from the four demos

Training-free

Score functions, tags, listwise ranking and agent pipelines all run on frozen weights.

Compute buys quality

Score functions gain +0.24 in search, a retrieval model tags images, and one call replaces an index.

New information drives gains

Patches add +0.37 mAP at no cost, crops add +0.075 at 14x, and calibration adds about zero.

Check out of domain

+0.24 in search became −0.016 on held-out tasks. Cheap score functions and private verifiers pass this check.

Test-time compute in Elasticsearch today

`retriever · rrf`

Fuse rankings

Fuse lexical and dense results with the `rrf` retriever, which the score-function search rediscovered.

`retriever · text_similarity_reranker`

Rerank the top-k

Rerank the top hits with a listwise model through the `text_similarity_reranker` retriever.

`field · semantic_text`

Score chunks

With `semantic_text` chunking, rank each document by its best chunk.

`agents · budget`

Budget the agent

Give agents a token budget, and test them on private multi-hop questions before raising it.

Search has always been test-time compute. Spend it where **new information** enters.



Thank you

Han Xiao, VP of AI, Elastic

